

Experiment No.	05
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study
GitHub Repository	https://github.com/The-F00L-00/ML_Lab
Student Name	R. Sundareswaran
Register Number	3122247001048
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 24, 2026

1 Objective

This experiment builds a Decision Tree classifier, then extends it into a Random Forest, and compares the two after tuning both with 5-Fold Cross-Validation. The main question I wanted to answer was how much an ensemble actually buys you over a single tuned tree, and which hyperparameters matter most for each.

2 Problem Statement

The task is to classify tumors in the Wisconsin Diagnostic Breast Cancer dataset as Malignant or Benign using 30 numerical features derived from digitized images of fine needle aspirates. A Decision Tree is trained and tuned first, followed by a Random Forest on the same train/test split, with both models optimized through 5-Fold Cross-Validation over a defined hyperparameter grid.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	https://archive.ics.uci.edu (via <code>sklearn.datasets.load_breast_cancer</code>)
Number of Samples	569
Number of Features	30
Number of Classes	2 (Malignant = 0, Benign = 1)
Missing Values	0
Train-Test Split	Train: 455 and Test: 114

4 Exploratory Data Analysis

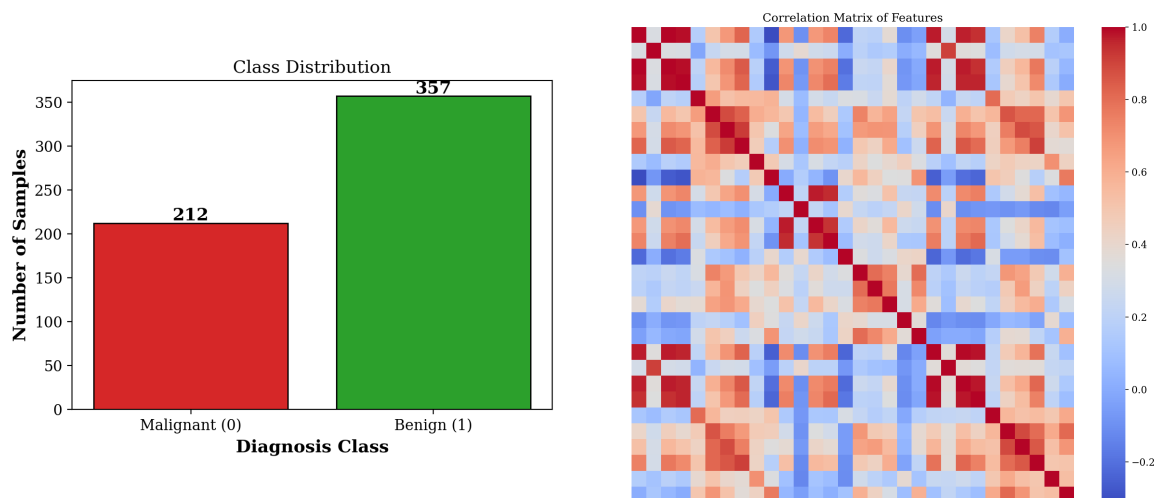


Figure 1: Class Distribution (left) and Feature Correlation Matrix (right)

Inference: Benign cases make up about 63% of the data and malignant cases the remaining 37%, a mild imbalance that stratified splitting handles well enough without resampling. The correlation heatmap shows what you’d expect anatomically: radius, perimeter, and area move together (both in their mean and “worst” variants) because they’re all measuring size, while texture and symmetry features are largely independent of these. That redundancy is actually a good sign for tree models, since a split on any one size feature captures most of what the others would offer.

5 Data Preprocessing

There were no missing values to handle, and the target was already binary-encoded (0 = Malignant, 1 = Benign). Features were standardized with `StandardScaler` mainly to keep the pipeline consistent across experiments, even though tree-based models don’t strictly need scaling. An 80:20 stratified split gave 455 training and 114 test samples while keeping the original class ratio intact, and no additional feature engineering was needed since the dataset already ships with extracted numerical features.

6 Machine Learning Algorithm

Working Principle

A Decision Tree splits the feature space at each node using the feature and threshold that most reduces impurity, measured by Gini Index or Entropy, until it hits a stopping rule like maximum depth or minimum leaf size. A Random Forest trains many such trees on bootstrapped samples of the data, restricts each split to a random subset of features so the trees don’t all learn the same thing, and then lets them vote on the final prediction.

Mathematical Formulation

Gini Index for a node with C classes:

$$G = 1 - \sum_{i=1}^C p_i^2$$

Entropy for the same node:

$$H = - \sum_{i=1}^C p_i \log_2 p_i$$

where p_i is the fraction of samples belonging to class i at that node, and the split chosen at each step is the one that maximizes the resulting drop in impurity. For the forest, the ensemble prediction is simply the majority vote, $\hat{y} = \text{mode}\{h_1(x), \dots, h_T(x)\}$, over the T individual trees.

Advantages and Limitations

Trees are easy to read and visualize and need almost no preprocessing, and forests build on that by cutting variance and generally overfitting less. The trade-off is that a single tree overfits easily once it grows deep, a forest is much harder to interpret than one tree, and both can lean too heavily on high-cardinality features.

7 Experimental Setup

Python Version	3.13.13
Scikit-Learn Version	1.7.5
IDE	Jupyter Notebook
Operating System	Fedora 42
Hardware	Intel Core i5 (11th gen), 8GB RAM

8 Hyperparameter Selection

Decision Tree (GridSearchCV)

Parameter	Value
criterion	gini
max_depth	5
min_samples_split	2
min_samples_leaf	4

Random Forest (GridSearchCV)

Parameter	Value
n_estimators	100
max_depth	10
max_features	log2
bootstrap	True

9 Results

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	0.9035	0.8444	0.9048	0.8736	0.9358
Random Forest	0.9561	0.9512	0.9286	0.9398	0.9924

Table 2: Test set performance (Malignant treated as the positive class)

Model	Train Accuracy	Test Accuracy
Decision Tree	0.9758	0.9035
Random Forest	1.0000	0.9561

Table 3: Train vs. test accuracy for both models

10 Result Visualization

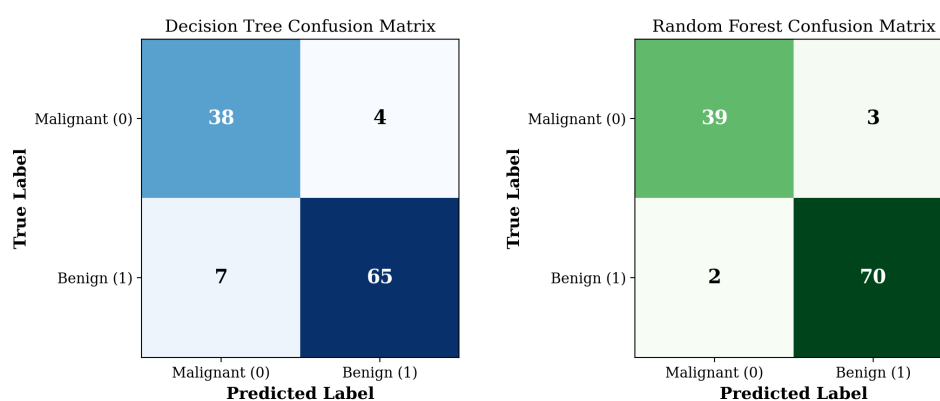


Figure 2: Confusion Matrices: Decision Tree (left) vs Random Forest (right)

Inference: The tree gets 11 of 114 test cases wrong; the forest gets only 5. What matters more than the raw count is where those errors land: the forest misses just 2 malignant cases as benign, compared to 4 for the tree, and a missed malignant diagnosis is exactly the kind of mistake you want to minimize here.

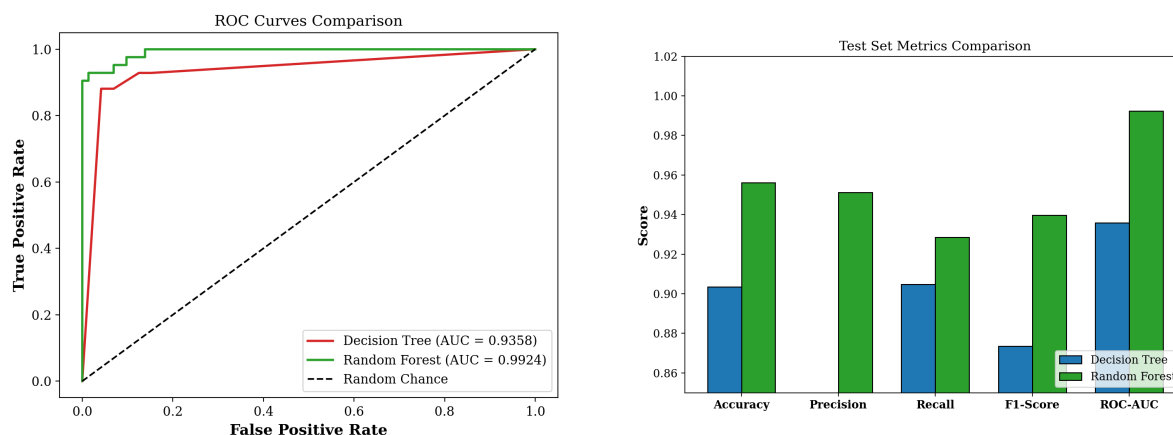


Figure 3: ROC Curves (left) and Test Set Metrics Comparison (right)

Inference: The forest’s ROC-AUC of 0.9924 beats the tree’s 0.9358, and its curve is visibly smoother – a natural consequence of averaging probability estimates over 100 trees instead of relying on one tree’s blocky, piecewise output. The bar chart on the right tells the same story across every metric: the forest wins on all five, with the biggest jump in precision.

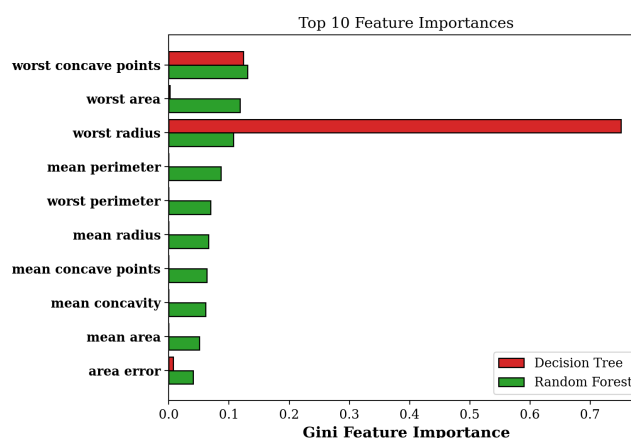


Figure 4: Top 10 Feature Importances: Decision Tree vs Random Forest

Inference: Both models lean on concave-points and perimeter/area features to separate the classes, which lines up with the intuition that larger, more irregularly-shaped masses are more likely to be malignant. The tree puts almost all its weight on a single top feature, while the forest spreads importance across more of them – a direct effect of the random feature subsampling at each split.

11 5-Fold Cross-Validation Performance Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	0.9451	0.9451	0.9121	0.9231	0.9670	0.9385
Random Forest	0.9670	0.9780	0.9341	0.9560	0.9890	0.9648

Table 4: 5-Fold CV accuracy on the tuned estimators

12 Discussion

Deeper trees fit noise, not signal – even at the tuned depth of 5, the tree’s train accuracy (97.58%) sits well above its test accuracy (90.35%), and letting depth grow unbounded made cross-validation scores worse, not better. For the tree, `min.samples.leaf` mattered most; every top-performing combination in the grid search fixed it at 4. For the forest, `max.features` = “log2” showed up consistently among the best combinations, which suggests decorrelating the trees counted for more than simply adding estimators. That decorrelation is also why the forest generalizes better despite hitting 100% training accuracy – averaging over many high-variance, low-bias trees cancels out their individual noise. Ensembling clearly paid off here, beating the tree on every metric, though that gain is largest precisely because the tree itself had so much variance to begin with.

13 Conclusion

Both models were tuned with GridSearchCV and 5-Fold Stratified Cross-Validation on the WDBC dataset. The Random Forest came out ahead across the board, reaching 95.61% test accuracy and a 0.9924 ROC-AUC against the Decision Tree’s 90.35% and 0.9358. Bagging plus random feature selection did what they’re supposed to do – they reduced the variance a single tree is prone to and produced smoother, more reliable probability estimates in the process.

14 References

1. Scikit-learn Developers, “Scikit-learn: Machine Learning in Python.” Available: <https://scikit-learn.org/stable/>
2. UCI Machine Learning Repository, Wisconsin Diagnostic Breast Cancer (WDBC) dataset. Available: <https://archive.ics.uci.edu>
3. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.